

Try to identify the classes you are creating

Before writing any code, we must understand the *problem we are trying to solve*.

Object-Oriented Design starts with **thinking**, not typing.

Get a Written Description of the Problem

Begin with a **clear written description** of the problem domain.

This can come from:

- A client
- An assignment prompt
- A real-world scenario
- Your own notes

 *If you cannot explain the problem in words, you are not ready to code it.*

Example Problem Description

“A bank system keeps track of customers, their accounts, and loans.

Customers can open accounts, deposit money, withdraw money, and apply for loans.

Tellers assist customers with these tasks.”

This description is our **starting point**.

Step 2: Identify the Nouns

Read the description carefully and **circle every noun** (including noun phrases and important pronouns).

These nouns are **candidates for classes**.

From the example:

- bank
- system
- customers
- accounts
- loans
- tellers
- money

Why Nouns Matter

In Object-Oriented Programming:

Nouns often become classes.

Verbs often become methods.

This is not a rule — but it is a *very strong guideline*.

Common Types of Classes

Tangible, physical items, such as vehicles, machines, or manufactured products

Roles fulfilled by people, including manager, employee, customer, teacher, or student

Outcomes of business activities, such as a customer order or a service quote

Information and record-keeping entities, including customer histories and payroll records

Refining the Noun List

After identifying all possible nouns, we must **refine the list**.

Not every noun should become a class.

Some nouns can be combined, removed, or simplified.

Nouns That Mean the Same Thing

Some nouns may **refer to the same concept**, even if different words are used.

These should usually be represented by **a single class**.

Example:

client and *customer*

employee and *staff member*

Choose one clear name and discard the rest.

Nouns We Do Not Need to Model

Some nouns appear in the description but are **not important to solving the problem**.

These nouns can usually be ignored.

Example:

system, process, task, screen

If the noun has **no data and no behavior**, it probably does not need a class.

Objects vs Classes

Some nouns describe **individual objects**, not general categories.

Classes represent **types of things**, not one specific instance.

Example:

this customer → object

Customer → class

Only nouns that represent a **general concept** should become classes.

Simple Values vs Classes

Some nouns represent **simple values**, not full objects.

These values can be stored in variables and **do not require a class**.

Example:

price, date, quantity, balance

If a noun can be represented as a single value (number or string), it usually does not need its own class.

Classes represent important concepts with data and behavior —
not every word in the description.

Step 3: Refine the List of Classes

Not every noun becomes a class.

Ask these questions:

- Does this thing have **data**?
- Does it have **behavior**?
- Does it need to be **represented independently**?

Refining the Example

Noun	Keep as Class?	Why
Bank	 Yes	Has customers, tellers, location
Customer	 Yes	Has data and actions
Account	 Yes	Stores balance, type
Loan	 Yes	Has amount, type
Money	 No	Data value, not an object
System	 No	Too abstract
Task	 No	Behavior, not a thing

Final Class List (Example)

After refining, we might keep:

- Bank
- Customer
- Account
- Loan
- Teller

These will become the **classes in our program**.

Identifying a Class's Responsibilities

Once the classes have been identified, the next step is to decide **what each class is responsible for**.

A class's responsibilities fall into two categories:

- what the class must **know**
- what the class must **do**

What a Class Must Know

The things a class must *know* become its **data attributes**.

Ask:

“What information does this class need to store?”

These values usually come from:

- the problem description
- real-world expectations

What a Class Must Do

The things a class must *do* become its **methods**.

Ask:

“What actions should this class be able to perform?”

These actions often appear as **verbs** in the problem description.

Turning Responsibilities into Code

- What a class must **know** → attributes
- What a class must **do** → methods

This is how we translate a real-world problem into an object-oriented design.

Asking the Right Questions (Very Important)

It is often helpful to ask:

- *In the context of this problem, what must the class know?*
- *In the context of this problem, what must the class do?*

Many responsibilities are stated directly in the problem description.

Others require **thought and design judgment**.

Applying This to Our Example

Now let's apply this process to the classes we have already identified.

(`StreetLight`, `Contact`, or the Bank example.)